Exact and matheuristic approaches for a real-world process selection and lot-sizing problem in the electrofused grains industry

José Roberto Dale Luche[a,*], Vitória Pureza[b], Reinaldo Morabito[b]

[a] *São Paulo State University (UNESP), Guaratinguetá, SP, Brazil*
[b] *Department of Production Engineering, Federal University of São Carlos (UFSCar), São Carlos, SP, Brazil*

Abstract

We revisit a real-world process selection and lot-sizing problem from the electrofused grains industry, in which each production process yields multiple products in fixed proportions and at most one process runs per period, fifteen years after it was reported intractable for exact methods. Under an equal-resources protocol with pre-registered hypotheses, we compare a modern MIP solver on the monolithic model, temporal-decomposition matheuristics (relax-and-fix construction with fix-and-optimize improvement, and a warm-started monolith), and a state-of-the-art hybrid GRASP-ILS metaheuristic with path relinking, on a new public benchmark of 60 instances spanning horizons from 19 to 190 periods. The verdict of the original study is reversed at the plant's own scale: instances derived from its order book now solve to proven optimality in seconds. We also isolate a modeling effect that governs difficulty – penalizing accumulated excess production at a thousandth of the weight of shortage moves four instance sets from routinely closed to largely open, holding everything else fixed. The pure metaheuristic, even with tenfold parallel resources, is dominated throughout by MIP-based methods. The resulting method-choice map has two axes: at equal budgets, decomposition wins the majority of instances from about 150 periods upward, while granting the monolith triple time restores its lead on most,

*Corresponding author.
Email address: ... (José Roberto Dale Luche)

though not all, of the largest instances – decomposition effectively buys one tier of computing budget, the currency of rolling re-planning. We release all instances, solutions, trajectories, and per-window logs, including dual signals, to support reproduction and learning-based extensions.

Keywords: Lot sizing, Process selection, Co-production, Matheuristics, Relax-and-fix, Fix-and-optimize, GRASP

---

1. Introduction

Electrofused grains, notably silicon carbide and fused aluminum oxide, are synthetic raw materials employed in the manufacture of abrasive, refractory, and ceramic products. Production starts in electric-resistance (Acheson-type) or electric-arc furnaces, whose output blocks are subsequently crushed, milled, and classified by sets of screens into a large number of granulometric ranges (grits) with strict physical–chemical specifications. A distinctive feature of the grain-finishing stage is *co-production in fixed proportions*: each configuration of the crushing and classification equipment – hereafter a *production process* – simultaneously yields a specific mix of final products in technologically determined proportions. Planners therefore do not decide production quantities product by product; they decide *which process to operate in each period*, and the product mix follows. Combined with lengthy setups (equipment reconfiguration and cleaning may consume an entire shift), a portfolio of dozens of products, and tight order-book due dates, this makes production scheduling a genuinely hard combinatorial task.

The problem was formalized by Luche et al. (2009) as a *process selection and lot-sizing problem* (PSP): a single-stage, big-bucket model in which at most one process may be assigned to each period and the objective is to minimize the total shortage of the order book (with a small penalty on excess production). At the time, the mixed-integer programming (MIP) model could not be solved to optimality within acceptable times for instances of practical size by the then state-of-the-art solver, which motivated constructive heuristics, local

search, and later a GRASP procedure (Luche, 2011). Two developments justify revisiting the problem fifteen years later. First, the computational landscape has changed on every axis at once: MIP solvers have improved by orders of magnitude through presolve, cut separation, primal heuristics, and parallel tree search (Koch et al., 2022; Achterberg and Wunderling, 2013; Achterberg et al., 2020), while commodity hardware has gained cores and clock speed. Whether a problem abandoned to heuristics in 2011 is still hard *today*, on a laptop, is a question of direct managerial relevance – independently of how the gain divides among its causes. Second, MIP-based construction and improvement heuristics – *relax-and-fix* (RF) and *fix-and-optimize* (FO), now standard components of the matheuristics toolbox (Sahling et al., 2009; Helber and Sahling, 2010; Toledo et al., 2015a) – have matured precisely on lot-sizing structures similar to the PSP, but have never been applied to it.

This paper reports a comprehensive computational study of the PSP that confronts three algorithmic paradigms under a strict equal-resources protocol: (i) the monolithic MIP model solved by a modern commercial solver; (ii) temporal-decomposition matheuristics (RF+FO and an RF-warm-started monolithic variant, RF+MIP); and (iii) a state-of-the-art hybrid metaheuristic combining reactive GRASP, iterated local search, variable neighborhood descent, tabu components, and cross-run path relinking (GRASP-ILS-PR). All methods run on the same machine, with the same wall-clock budgets and explicit thread accounting, and the central comparisons were specified as pre-registered hypotheses before the experiments were executed. Rather than asking only *whether* a heuristic beats the solver, we map *when* each method is the tool of choice as a function of instance scale (planning horizons from $T = 19$ to $T \approx 190$ periods) and available computing time (from operational re-planning budgets of 5–10 minutes to tactical budgets of 1–3 hours). The resulting map shows the monolith dominant up to twice the industrial horizon, decomposition winning the majority of instances from roughly 150 periods upward at equal budgets, and the monolith recovering most – not all – of that ground when granted triple time.

The contributions are fourfold. First, we isolate a modeling effect that governs the problem's difficulty: penalizing accumulated excess production – an inventory-cost proxy worth a thousandth of a unit of shortage – turns a formulation that a modern solver closes routinely into one that it leaves open on most instances of the same size. The comparison is controlled (same solver, machine, threads, budget, and instances; only the objective changes), and it qualifies a broader practice: secondary objective terms that look innocuous in a model can dominate its computational character. Second, we develop and specify, at a reproducibility level rarely reported (bound-update mechanics, per-window time-limit policies, and budget guards included), RF and FO matheuristics tailored to the co-production structure of the PSP, together with an RF+MIP hybrid that trades a modest construction budget for a strong initial incumbent of the monolithic solve. Third, we provide what is, to the best of our knowledge, the first publicly available benchmark for lot sizing with co-production in fixed proportions: 60 instances spanning ten problem scales, with best-known solutions, dual bounds, and full per-run trajectories, released in open formats. Fourth, we derive managerial insights for the electrofused grains mill, including the practical consequence that, at the scale of the plant's order book, scheduling can now be re-optimized *exactly* within any daily re-planning deadline – by the solver itself – while decomposition becomes the operational tool only at horizons several times longer.

The remainder of the paper is organized as follows. Section 2 reviews the related literature and positions the PSP within the lot-sizing and scheduling taxonomy. Section 3 describes the industrial setting and the mathematical model. Section 4 presents the RF, FO, and RF+MIP methods, and Section 5 summarizes the hybrid GRASP-ILS-PR baseline. Section 6 reports the experimental protocol and results, and Section 7 concludes.

2. Related work

We review four streams: the lot-sizing and scheduling taxonomy in which the PSP is embedded (§2.1); applications featuring process selection and co-production in fixed proportions (§2.2); MIP-based matheuristics of the relax-and-fix and fix-and-optimize families (§2.3); and the solver-progress and experimental-methodology literature that underpins our protocol (§2.4).

2.1. Lot sizing and scheduling: taxonomy and reviews

Dynamic lot-sizing models that integrate sequencing decisions are classically organized by their time structure. Small-bucket models allow at most one (or a few) setups per period: the discrete lot-sizing and scheduling problem (DLSP; Fleischmann, 1990) imposes all-or-nothing production, the continuous setup lot-sizing problem (CSLP; Karmarkar and Schrage, 1985) relaxes full capacity usage, and the proportional lot-sizing and scheduling problem (PLSP; Drexl and Haase, 1995) allows the setup state to change once within a period. The general lot-sizing and scheduling problem (GLSP; Fleischmann and Meyr, 1997) subsumes these structures through micro-periods, and the capacitated lot-sizing problem with sequence-dependent setups (Haase, 1996) extends the big-bucket tradition. Comprehensive surveys trace the evolution of this family: Drexl and Kimms (1997) and Karimi et al. (2003) for the foundations, Jans and Degraeve (2008) for industrial extensions, Buschkühl et al. (2010) for solution approaches, Glock et al. (2014) at the tertiary level, and, most relevant to our positioning, the classification of simultaneous lot-sizing and scheduling models by Copil et al. (2017), recently extended to secondary resources by Wörbelauer et al. (2019). Within this taxonomy the PSP is a single-stage, single-machine, big-bucket model with at most one "product" – in fact, one *process* – per period; its distinctive trait, absent from the standard classes, is that the assignment decision releases a technologically fixed *vector* of products rather than a single item. Demand is met against cumulative order-book quantities with shortage minimization, which relates the model to service-level-oriented variants rather than to the cost-minimization mainstream (Pochet and Wolsey, 2006).

2.2. Process selection and co-production in fixed proportions

The PSP was introduced in the electrofused grains context by Luche et al. (2009), who combined process selection with single-stage lot sizing, proposed a constructive heuristic, and reported that instances of practical size defeated the MIP technology of the time. A structurally similar coupling of process configuration, lot sizing, and scheduling arises in the molded pulp industry (Martínez et al., 2018), where each mold configuration also determines a mix of items, and in small foundries, where each furnace alloy enables a set of castable parts (de Araujo et al., 2008). Related recipe- or process-based settings include animal nutrition plants (Toso et al., 2009; Clark et al., 2010), chemical plants in which one item may be produced by alternative processes (TODO, 2021a), and integrated soft-drink production where tank lots feed bottling lines (Ferreira et al., 2009; Toledo et al., 2015b). Co-production in fixed (or stochastic) proportions is also the defining feature of semiconductor planning, where each wafer lot "bins" into multiple performance grades (Leachman, 2002; Ng et al., 2007), and connects to multi-period cutting-stock models in which a cutting pattern plays the role of a process (Silva et al., 2023). Across this literature, exact approaches are typically abandoned at realistic scales in favor of tailored heuristics; none of these works, however, revisits the abandoned monoliths with current solver technology, and public benchmarks for fixed-proportion co-production are, to the best of our knowledge, unavailable.

2.3. Relax-and-fix and fix-and-optimize matheuristics

Relax-and-fix constructs solutions by partitioning the integer variables – most naturally along the time axis – and solving a sequence of MIPs in which one window is kept integral, earlier windows are fixed, and later windows are relaxed (Dillenberger et al., 1994; Pochet and Wolsey, 2006). Fix-and-optimize (also called exchange or MIP-based improvement) iteratively re-optimizes subsets of integer variables while fixing the remainder at incumbent values; it was popularized for multi-level capacitated lot sizing by Sahling et al. (2009) and Helber and Sahling (2010). The combination of the two – RF for construction,

FO for improvement – was systematized by Toledo et al. (2015a) on multi-level lot-sizing and glass-container problems, and industrial variants have since accumulated: integrated pulp and paper planning (Santos and Almada-Lobo, 2012), later improved by variable neighborhood search (Figueira et al., 2013) and by structure-exploiting matheuristics (Furlan et al., 2024); beverage plants with temporal cleanings (Toscano et al., 2020); and perishable-product lines, where relax-and-fix also supplies competitive dual bounds (TODO, 2021b). Iterative MIP-based neighborhood searches for lot sizing and scheduling were explored by James and Almada-Lobo (2011), and the genre is surveyed under the "matheuristics" label by Fischetti and Fischetti (2018). Our RF+FO follows the time-window template of Toledo et al. (2015a); the differentiating elements are the co-production structure (windows re-optimize the process schedule, with all product balances linked through cumulative demand), a fully specified budget-control policy whose implementation details we report for reproducibility, and the RF+MIP variant in which the RF solution warm-starts the *monolithic* model – a hybrid whose value, as our experiments show, depends on whether the incumbent it buys is worth the branch-and-bound time it costs.

2.4. Solver progress and experimental methodology

Machine-independent measurements document speedups of several orders of magnitude in MIP solving between the early 2000s and 2020, driven by presolve, cutting planes, primal heuristics, and parallel search (Koch et al., 2022; Achterberg and Wunderling, 2013; Achterberg et al., 2020; Bixby, 2012). This progress reframes older intractability findings: conclusions drawn against CPLEX 11–12 – as in the original PSP studies – need not hold against CPLEX 22, and revisiting them yields calibrated evidence of practical value. On the methodology side, our experimental protocol adopts community standards for heuristic–exact comparisons: performance profiles (Dolan and Moré, 2002), time-to-target plots for randomized methods (Aiex et al., 2007), and public release of instances and solutions in the spirit of MIPLIB (Gleixner et al., 2021). The metaheuristic baseline itself follows established components: GRASP (Feo and Resende,

1995) with reactive parameter control (Prais and Ribeiro, 2000), iterated local search (Lourenço et al., 2010), and path relinking (Laguna and Martí, 1999; Resende and Ribeiro, 2016). Finally, the managerial framing of short re-planning budgets draws on the rolling-horizon tradition initiated by Baker (1977).

Gap. In summary: (i) the PSP with co-production in fixed proportions has not been revisited since 2011, despite solver progress that plausibly changes its practical status; (ii) matheuristic studies seldom report primal–primal comparisons against a modern solver under strictly equal wall-clock and thread budgets, leaving the "method of choice" question anecdotal; and (iii) no public benchmark exists for this problem class. This paper addresses all three gaps, with hypotheses registered before experimentation.

3. The process selection and lot-sizing problem

3.1. Industrial setting

We study the grain-finishing stage of a manufacturer of electrofused grains (silicon carbide and fused aluminum oxide). Upstream, raw materials are fused in electric furnaces and cooled into blocks of roughly twenty tonnes; the finishing stage crushes, mills, and screens this material into final products defined by granulometric ranges and physical–chemical specifications. The finishing equipment can be arranged in a number of alternative configurations – the *production processes*. Two features of these processes drive the planning problem. First, *co-production in fixed proportions*: operating process j for one period yields a technologically determined quantity a_{ij} of every product i simultaneously; the planner selects processes, not product quantities. Second, *setup magnitude*: reconfiguring and cleaning the equipment between processes consumes a substantial fraction of a planning period, which motivates a big-bucket representation in which at most one process is operated per period. Demand stems from an order book with due dates; the commercial priority is meeting the order book, so the objective minimizes total shortage, with a small penalty discouraging

gratuitous excess production (excess is held as inventory and may serve future orders, but ties up capital and storage).

3.2. Mathematical model

Let I denote the set of products, J the set of processes, and $T = \{1, \ldots, |T|\}$ the set of periods. Parameter $a_{ij} \geq 0$ gives the output of product i when process j is operated for one period, and $d_{it} \geq 0$ the demand of product i due at period t. The decision variables are $x_{jt} \in \{0, 1\}$, equal to 1 if process j is operated in period t, and the nonnegative continuous variables f_{it} and e_{it}, which measure the deviation of cumulative production from cumulative demand of product i up to period t: f_{it} is the backlog and e_{it} the physical inventory at the end of that period. Summed over periods, the two terms of (1) are therefore the classical backlog-periods and inventory-periods of dynamic lot sizing (Pochet and Wolsey, 2006), and γ is a holding cost expressed as a fraction of the unit backlog cost. The model, referred to as MFP (minimization of shortage of production, after Luche et al., 2009), reads:

$$\min \quad \sum_{i \in I} \sum_{t \in T} \left(f_{it} + \gamma \, e_{it} \right) \tag{1}$$

$$\text{s.t.} \quad \sum_{j \in J} \sum_{\tau \leq t} a_{ij} \, x_{j\tau} + f_{it} - e_{it} \; = \; \sum_{\tau \leq t} d_{i\tau}, \qquad i \in I, \; t \in T, \tag{2}$$

$$\sum_{j \in J} x_{jt} \; \leq \; 1, \qquad t \in T, \tag{3}$$

$$x_{jt} \in \{0, 1\}, \quad f_{it}, \, e_{it} \geq 0, \qquad i \in I, \; j \in J, \; t \in T. \tag{4}$$

Constraints (2) balance cumulative production against cumulative demand for every product and period, so that f_{it} captures lateness with respect to due dates (not only end-of-horizon shortage), and constraints (3) allow at most one process – possibly none – per period. The weight $\gamma = 0.001$ makes excess lexicographically subordinate to shortage in practice. The holding term is a deliberate refinement of the original formulation (Luche et al., 2009), which minimized backlog alone under an inequality balance and therefore priced early production at zero. Charging it, even lightly, matters industrially – grain inventories

Table 1: Benchmark dimensions by instance family.

Set	Inst.	T	J	I	Binary vars.	Continuous vars.	Constraints
Real	10	19–25	159–160	50	3 021–4 000	1 901–2 501	970–1 276
2X	10	38	159	50	6 042	3 801	1 939
3X	10	57	159	50	9 063	5 701	2 908
4X	10	76	159	50	12 084	7 601	3 877
5X	10	95	159	50	15 105	9 501	4 846
8X	5	152	159	50	24 168	15 201	7 753
10X	5	190	159	50	30 210	19 001	9 691

tie up capital and covered storage – and, as Section 6.4 shows, it is what governs the computational character of the model. The model has $|J|\,|T|$ binary and $2|I|\,|T|$ continuous variables; for the largest instances considered here ($|J| = 159$, $|T| = 190$) this amounts to 30,210 binaries. Table 1 summarizes the dimensions of the full benchmark family.

Model MFP is a single-stage, big-bucket member of the simultaneous lot-sizing and scheduling family (Copil et al., 2017), with an assignment-type setup structure per period; its distinctive element is that each period's assignment releases the fixed output vector $(a_{ij})_{i \in I}$, coupling all products through constraints (2).

3.3. Instances and empirical difficulty

Our benchmark derives from a single real order book of the plant ($|J| = 159$ processes, 50 items, 19 daily periods). From it, the original study generated ten instances by redistributing each item's total demand at random over the horizon, preserving aggregate volumes; we inherit this set (denoted Real, $|T| \in \{19, 25\}$, one instance using a longer reserved horizon) and the horizon-replicated sets obtained by the rule documented in the original study (Luche, 2011, Section 6.2.2): the number of periods is multiplied by k and the demand of the original horizon is repeated exactly in each 19-period block, yielding sets 2X–5X ($k \in \{2, \ldots, 5\}$, inherited) and the new sets 8X and 10X introduced in this paper ($k \in \{8, 10\}$, $|T|$ up to 190). Section 6.1 details the generation and its validation against the

inherited sets. Demand is therefore periodic by construction across the whole family – a property we state explicitly because it keeps the generation rule constant along the scale axis, avoiding confounds; empirically, periodicity does not make the instances easy for the solver (Section 6).

Two empirical observations shape the experimental design. First, difficulty is heterogeneous within sets: the subclass

3X: IncT3x_1, IncT3x_3, IncT3x_7, IncT3x_9, IncT3x_10;
4X: IncT4x_1, IncT4x_3, IncT4x_7, IncT4x_9, IncT4x_10;
5X: IncT5x_1, IncT5x_3, IncT5x_7, IncT5x_9, IncT5x_10

exhibits both above-median solver gaps and above-median BKS-to-bound gaps within its set – on IncT5x_10, the best known solution still lies about 17% above the three-hour dual bound, so the true optimality gap is unknown. Second, the solver's parallel search, although deterministic for a fixed budget, is budget-nonmonotone in the primal sense: on three instances the one-hour run returned a better incumbent than the independent three-hour run. Both observations motivate reporting gaps against best-known solutions (BKS), defined as the minimum objective over *all* methods, budgets, and seeds considered in this study, rather than against dual bounds alone.

4. Matheuristics

We propose temporal-decomposition matheuristics that use the MIP solver as a subroutine over restricted versions of model (1)–(4). A *schedule* is a vector $s \in (\{0\} \cup J)^{|T|}$ assigning to each period a process or idleness; any schedule is feasible, and its cost is computed by an evaluator that replicates the MIP objective exactly (the evaluator reproduces the solver objective on reference solutions to within 10^{-2}, a consistency check enforced throughout).

4.1. A single restricted model with bound control

Relax-and-fix requires period-dependent variable regimes (fixed, binary, or linearly relaxed), whereas algebraic modeling systems attach integrality to whole

variables. We therefore build, once per instance, a single model with two variable families: binary x_{jt} and continuous $\bar{x}_{jt} \in [0,1]$, every occurrence of the assignment variable in (2)–(3) being replaced by $x_{jt} + \bar{x}_{jt}$. Period regimes are then governed purely by bounds: a *relaxed* period fixes $x_{jt} = 0$ and frees $\bar{x}_{jt}$; a *binary* period frees x_{jt} and fixes $\bar{x}_{jt} = 0$; a *fixed* period sets x_{jt} to the incumbent value and $\bar{x}_{jt} = 0$. Between consecutive solves only bounds change, so the model is generated once. An implementation detail proved decisive: updating bounds record-by-record dominated the running time, whereas batch record updates reduced the construction phase from hundreds of seconds to below ten on the largest instances. We report this because the practicality of temporal decomposition in algebraic modeling environments hinges on it.

4.2. Relax-and-fix construction

The construction sweeps time windows of width σ advancing by $\delta < \sigma$ periods (overlap $\sigma - \delta$), the last window anchored at the end of the horizon so that binary windows cover $1..|T|$. At each iteration the current window is binary, all earlier periods are fixed at the incumbent partial schedule (only the first δ periods of the previous window are fixed definitively; the overlap is re-optimized), and all later periods are relaxed. Each window MIP is solved with relative gap tolerance 10^{-3} and a dynamic time limit $\ell = \max\{5,\ \min\{\ell_{\mathrm{RF}},\ B^{\mathrm{rem}}_{\mathrm{RF}}/w^{\mathrm{rem}},\ B^{\mathrm{rem}}\}\}$ seconds, where ℓ_{RF} caps the per-window limit, $B^{\mathrm{rem}}_{\mathrm{RF}}$ and B^{rem} are the remaining construction and global budgets, and w^{rem} the number of windows still to solve. The construction budget is $B_{\mathrm{RF}} = \min\{0.25\,B,\ w\,\ell_{\mathrm{RF}}\}$ for global budget B and w windows. If a window solve yields no integer incumbent, its limit is doubled once; on a second failure – and whenever the global budget guard triggers – all undecided periods are completed by a greedy rule based on the evaluator, so the construction never returns an incomplete schedule. Under very short budgets, where the per-window allowance drops below a threshold ($B_{\mathrm{RF}}/w < 10$ s), windowed construction is skipped altogether in favor of the greedy rule, and the entire budget is transferred to the improvement phase. In addition, a wall-clock guard charges all per-window overhead to the construction

budget (early implementations that charged only solver time silently overran it by a factor above two on the longest horizons), and every run computes the greedy solution regardless, the improvement phase starting from the better of the two constructions (*greedy floor*).

4.3. Fix-and-optimize improvement

Improvement re-optimizes windows of ω consecutive periods: binaries inside the window are freed, all others are fixed at the incumbent, and the window MIP is solved with relative gap tolerance 10^{-2}, per-window limit ℓ_{FO}, and the incumbent supplied as a MIP start (accepted starts were verified in the solver log). A solution is accepted if it improves the global objective by more than 10^{-6}; since the start is feasible for every window subproblem, accepted moves never degrade the incumbent, and the method reports the best solution found in *any* phase of the run. Windows first sweep the horizon with step $\omega/2$; after a full sweep without improvement, contiguous windows with random start positions (seeded) are drawn until the budget is exhausted. Parameters were tuned on the two hardest 5X instances over the grid $\omega \in \{12, 20, 28\} \times \ell_{\mathrm{FO}} \in \{30, 60\}$ (Section 6 discloses this choice and its implications), selecting $\omega = 20$, $\ell_{\mathrm{FO}} = 30$ s: on $|T| = 95$, $\omega = 12$ proved too myopic (mean gap $+7.2\%$ to BKS) while $\omega = 20$ attained -1.1%, i.e., new best-known solutions.

4.4. Warm-started monolith (RF+MIP)

The third method uses relax-and-fix only to produce a high-quality integer solution quickly (on the order of seconds to two minutes), then solves the *monolithic* model with the remaining budget, zero gap tolerance, and the RF solution as a MIP start. Relative to the cold-started solver under the same wall-clock budget, RF+MIP sacrifices a fraction of branch-and-bound time in exchange for a strong initial incumbent; unlike RF+FO, it retains the solver's global view and dual bound. Because the construction consumes budget, RF+MIP is not dominant by construction: a post-mortem on one 10X instance showed a poor construction consuming over half the budget and an accepted – but unhelpful –

Algorithm 1 RF+FO (sketch; RF+MIP replaces lines 6–9 by one warm-started solve of the full model)

1: build restricted model with bound control (§4.1)
2: **if** $B_{\mathrm{RF}}/w < 10$ s **then** $s \leftarrow$ greedy construction
3: **else** $s \leftarrow$ relax-and-fix sweep (§4.2)
4: **end if**
5: $s^* \leftarrow s$
6: **while** budget remains **do**
7: choose next window (sweep, then random after a no-improvement sweep)
8: fix all periods outside the window at s^*; free the window; MIP-start at s^*
9: solve with limit ℓ_{FO}, tolerance 10^{-2}
10: **if** improvement $> 10^{-6}$ **then** update s^*
11: **end if**
12: **end while**
13: **return** best schedule found in any phase

MIP start, motivating the wall-clock guard and greedy floor above. Empirically, RF+MIP is the best variant at 8X, where the solver's global view still pays, while at 10X the monolithic model itself becomes the bottleneck and window-based FO improvement scales better (Section 6.7).

4.5. Parameter summary

Table 2 summarizes the parameter values used in the final campaign.
All runs record full provenance (solver version, threads, hardware, seeds) and per-window logs (window position, wall time, incumbent profile, and post-solve duals of (2) and (3)); the logs are released with the benchmark.

5. Hybrid metaheuristic baseline

To situate the matheuristics against the strongest *pure* metaheuristic we could build – one that never calls a MIP solver – we developed a hybrid pro-

Table 2: Matheuristic parameters (final values).

Phase	Parameter	Value	Meaning
RF	σ / δ	10 / 5	window width / step
RF	ℓ_{RF}	120 s	per-window cap
RF	optcr	10^{-3}	window gap tolerance
RF	B_{RF}	$\min\{0.25B,\, w\ell_{\mathrm{RF}}\}$	construction budget
FO	ω / step	20 / 10	window width / sweep step
FO	ℓ_{FO}	30 s	per-window cap
FO	optcr	10^{-2}	window gap tolerance
all	threads	0 (all cores)	explicit solver threads

cedure (GRASP-ILS-PR) combining components with strong track records in the GRASP literature. We summarize it here and refer to the released source code for full detail.

Solutions are schedules $s \in (\{0\} \cup J)^{|T|}$ evaluated by the same exact evaluator used by the matheuristics (Section 4), so all methods optimize literally the same objective. Construction is a reactive GRASP (Feo and Resende, 1995; Prais and Ribeiro, 2000): candidate processes for each period are ranked by myopic shortage reduction, the restricted candidate list parameter α being drawn from a pool whose probabilities adapt to observed solution quality. Local search is a variable neighborhood descent over process-exchange and Or-opt-style relocation moves on the schedule; for long horizons, moves are restricted by a sliding time window with a tabu mechanism to keep iteration costs bounded. The improvement loop is an iterated local search (Lourenço et al., 2010) whose perturbation applies multi-bridge reinsertions of schedule segments, with adaptive strength. Ten workers run in parallel from different seeds and share an elite pool through cross-run path relinking (Laguna and Martí, 1999; Resende and Ribeiro, 2016), and the incumbent trajectory of every worker is logged.

Under the equal-resources protocol of Section 6, GRASP-ILS-PR occupies a deliberately generous position: its ten workers exploit the full machine for the

whole wall-clock budget, and its per-instance result is the best of ten runs. This makes the comparison against the matheuristics and the solver *a fortiori*: any deficit observed for GRASP-ILS-PR cannot be attributed to a resource handicap. Historically, this procedure is the modernized descendant of the heuristics proposed for the PSP in the original studies (Luche et al., 2009) (Luche, 2011), and it improves substantially upon them: relative to its own previous version, median best-objective improvements are 20.3% on 2X, 33.4% on 3X, 41.2% on 4X, 43.9% on 5X, and 6.5% on Real; across the large 3X–5X sets the median improvement is 39.0%. Its role in this paper, however, is that of a calibrated baseline representing the pure-metaheuristic paradigm.

6. Computational experiments

6.1. Benchmark and public release

The benchmark comprises 60 instances in seven sets. All of them descend from a single real order book of the plant ($|J| = 159$ processes, 50 items, 19 daily periods). Set Real contains the ten instances obtained in the original study by redistributing each item's total demand at random over the horizon ($|T| \in \{19, 25\}$, $|J| \in \{159, 160\}$, $|I| \in \{50, 52\}$; one instance uses a longer reserved horizon). A consequence worth stating explicitly is that *no released instance reproduces the plant's actual order book*: demand profiles are randomized derivatives of it, which is what makes the public release compatible with the confidentiality of the company's commercial data while preserving the technological structure – the process–product matrix (a_{ij}) and the aggregate volumes – that makes the benchmark realistic. Sets 2X–5X, inherited from the original study, and the new sets 8X and 10X extend the horizon by the replication rule of Section 3.3 ($|T| = 19k$ for factor k; five instances each for 8X and 10X, built on the $|T| = 19$ order-book bases). We reimplemented the generator and validated it by regenerating the inherited sets: for the 36 instances whose 19-period base is available, the regenerated files match the inherited ones record by record (demand, horizon, products, and technology matrix); the four instances indexed

_1 descend from a base not preserved in our archives and are inherited as-is, outside the generator's correspondence. The generator is deterministic; the manifest records execution provenance only. All instances, solutions, per-run trajectories, and per-window logs are released at .

6.2. Protocol

All experiments ran on a single machine (AMD Ryzen 7 5800H, 8 cores / 16 threads, 32 GB RAM) with CPLEX 22.1.2 called through GAMSPy, deterministic parallel mode, and threads set explicitly to all cores for every method, including window subproblems. Methods are compared under equal wall-clock budgets – 600, 3,600, and 10,800 s – with all model-generation and bookkeeping overhead charged to the budget. The GRASP-ILS-PR baseline runs ten parallel workers per instance and reports the best of ten, a deliberately generous allocation (Section 5). Best-known solutions (BKS) are the minimum objective over all methods, budgets, and seeds, subject to a consistency safeguard that rejects any candidate below an available dual bound; this safeguard exposed evaluator inconsistencies in the legacy heuristic of the original study, whose results we therefore exclude from comparison tables (they remain in the released raw data, flagged). The central comparisons were pre-registered as hypotheses H1–H4 before the corresponding experiments, and a cross-budget question Q5 was registered before the final solver runs; we report all verdicts as registered, separating clearly labeled post-hoc analyses. Matheuristic parameters were tuned on the two hardest 5X instances (Section 4.3); both belong to the evaluation set, a choice we disclose and weigh when interpreting results on those two instances. Unless stated otherwise, all results refer to the model with $\gamma = 0.001$; the $\gamma = 0$ variant appears only in Section 6.4.

6.3. The problem's practical status today

The original study concluded that instances of practical size defeated the MIP technology available at the time, which is what motivated fifteen years of heuristic development for this problem (Luche et al., 2009). That conclusion

no longer holds at the scale of the plant. With CPLEX 22 on a current laptop, every instance of the Real set is solved to proven optimality in seconds (mean wall time 8 s), and every instance of 2X within the one-hour budget (mean 687 s, all proven optimal). Difficulty reappears only from 3X upward: at three hours, mean gaps grow from 1.7% (3X) to 3.9% (4X) and 7.2% (5X), with no proven optima in those three sets.

We deliberately refrain from attributing this reversal to solver progress alone. Between the original experiments and ours, the solver version, the machine, the number of threads, and – as Section 6.4 shows – the objective function itself all changed; isolating any single factor would require the machine-independent protocols of dedicated studies (Koch et al., 2022; Achterberg and Wunderling, 2013), which are outside our scope. The statement we do make is about practice rather than causes, and it is the one that matters for the plant: what once justified building heuristics is now dispatched by a general-purpose solver within an operational deadline.

Two byproducts of our own campaign qualify how solver output should be read at the larger scales. Incumbents are budget-nonmonotone: on three instances the one-hour run returned a better solution than the independent three-hour run, a consequence of different search trajectories rather than of nondeterminism, since the parallel mode is deterministic. And dual bounds are weak on a hard subclass – on IncT5x_10 the best known solution still lies about 17% above the three-hour bound – so true optimality gaps there are unknown, which is why we report gaps against best-known solutions.

6.4. What the excess penalty costs

The model of Section 3.2 charges accumulated excess production at $\gamma = 0.001$ per unit and period; the original formulation minimized shortage alone. Because everything else can be held fixed – same instances, solver, machine, threads, tolerance, and time limit – the two variants isolate the computational cost of that single term. Table 3 reports both on the 40 instances of sets 2X–5X at the three-hour budget.

Table 3: Controlled effect of removing the excess penalty ($\gamma = 0$).

Set	Inst.	Opt. ($\gamma = 0$)	Gap $\gamma = 0$ (%)	Gap $\gamma = 0.001$ (%)	Time $\gamma = 0$ (s)	Time $\gamma = 0.001$ (s)	ΔZ	Δ (%)
2X	10	10	0.00	0.00	24.9	666.5	5 579.7	19.16
3X	10	10	0.00	1.69	47.7	10 846.3	8 612.0	25.09
4X	10	10	0.00	3.91	114.2	10 831.9	11 641.4	29.78
5X	10	10	0.00	7.21	178.2	10 821.6	15 202.2	34.19

The contrast is sharp: all 40 $\gamma = 0$ runs close to proven optimality, with mean wall times from 24.9 s (2X) to 178.2 s (5X), whereas the $\gamma = 0.001$ model closes only the ten 2X instances within the same three-hour limit and reaches mean residual gaps of 1.69%, 3.91%, and 7.21% on 3X, 4X, and 5X. The controlled sanity relation $Z(\gamma = 0) \leq Z(\gamma = 0.001)$ holds on every instance.

The mechanism is structural rather than numerical. Under $\gamma = 0$, once cumulative demand is covered any additional output is free, so the feasible region contains large plateaus of equivalent optimal schedules and the solver prunes aggressively. Under $\gamma > 0$, each of those schedules carries a different accumulated excess, the plateaus collapse into a single optimum, and the linear relaxation – which can spread production fractionally across processes at negligible excess – becomes markedly weaker. The practical lesson generalizes beyond this application: a secondary objective term introduced for modeling realism, at a weight three orders of magnitude below the primary criterion, can decide whether an industrial lot-sizing model is routinely solvable or requires the decomposition machinery of the following sections.

6.5. Main comparison at the one-hour budget

Table 4 reports, for all 60 instances at 3,600 s, the cold-started solver, RF+FO, RF+MIP, and the GRASP-ILS-PR baseline. Three findings organize the section. First, the pure metaheuristic is dominated everywhere: GRASP-ILS-PR best-of-ten trails the solver's primal by 12–26% on average on 3X–5X, and trails RF+FO on every large instance, despite its tenfold parallel resources – evidence that, on this problem class, MIP-based components are not optional refinements but the main engine. Second, up to 5X ($|T| \leq 95$) the monolithic solver remains

the method of choice at this budget: it wins the per-instance majority in every set (10–0 on Real and 2X; 9–1 on 3X and 4X; 6–4 on 5X, where RF+FO takes the four hardest instances and sets a new BKS on IncT5x_10, 4.6% below the solver's three-hour incumbent). Third, robustness: across seeds (grade C), RF+FO shows zero variance on five of twelve replicated instances and coefficients of variation below 4% elsewhere, with window-log audits confirming that different seeds explore different window sequences (Table 6). Table 5 reports the paired Wilcoxon tests behind the pooled comparisons.

6.6. Operational budgets

At 600 s – the regime of daily re-planning – the picture is scale-dependent (Table 7). At the plant's own scale the question is moot: the solver alone closes the instances in seconds, so the managerial conclusion is direct and positive: order-book scheduling that was heuristic territory in the original study is now re-optimized exactly within any operational deadline. On the replicated scales the decomposition methods become competitive from 3X upward (4–5 of 10 wins per set for RF-based methods) and majority winners at 8X (RF+MIP, 3/5) and 10X (RF+FO, 4/5). Aggregated over all 60 instances the paired difference between RF+FO and the cold solver is not significant (median +0.12%, $p = 0.27$), the small-instance cells masking the large-scale advantage – which is precisely why we report the frontier by cell rather than pooled. Against the truncated GRASP-ILS-PR trajectories, RF+FO wins all six large pilot instances at 600 s with median advantage above 20% ($p = 0.031$). The registered hypothesis H1 (decomposition beats the cold solver in $\geq 4/6$ pilot instances at both 300 and 600 s) is *false* as registered (2/6 and 3/6): the original construction policy starved the improvement phase at 300 s on $|T| = 95$. The post-hoc corrected policy (greedy construction when the per-window allowance falls below 10 s) repairs the failure cases – all four regression cells now beat the GRASP-ILS-PR reference – but we keep the registered verdict and label the repair as post-hoc.

6.7. Scale-up and the method-choice frontier

The new 8X and 10X sets ($|T| = 152$ and 190; up to 30,210 binaries) locate the frontier. At 3,600 s the decomposition methods win the majority in both sets – RF+MIP at 8X (3/5) and RF+FO at 10X (3/5), with paired median advantage of 4.5% for RF+FO over the cold solver ($n = 10$, $p = 0.19$, rank-biserial -0.49): consistent in direction, not statistically significant at this sample size. Within the decomposition family the two variants change places with scale: at 8X the warm-started monolith exploits the solver's global view, while at 10X the model itself becomes the bottleneck and window-based improvement scales better – on IncT10x_4, RF+MIP stalls at 376,198 while RF+FO, from the same construction, reaches 172,983. Construction diagnostics temper the story at the extreme scale: at $|T| \geq 152$ the windowed relax-and-fix could not beat the greedy floor within its wall budget, so all corrected runs start from the greedy solution and the improvement phase does the heavy lifting; better construction at extreme scale is an open problem we return to in Section 7.

The registered cross-budget question completes the map. Q5 asked whether the cold solver, given 10,800 s, strictly beats the best decomposition result at 3,600 s; the answer is *yes* in both sets (4/5 at 8X, 3/5 at 10X), improving six best-known solutions in the process (Table 9). Two 10X instances resist even the tripled budget (IncT10x_2 by 14.5% and IncT10x_6 by 2.1%). The synthesis – Table 8 and Figure 4 – is a method-choice map with two genuine axes: at equal budgets, decomposition dominates from $|T| \approx 150$ upward; granting the monolith one budget tier (3×) restores its lead at 8/10 of the largest instances. Decomposition, in other words, buys approximately one tier of solver time – exactly the currency that matters in rolling re-planning. We disclose the asymmetry that decompositions were not executed at 10,800 s; the released code and instances allow closing that cell. Table 10 reports the construction source and remaining improvement time for the corrected scale-up runs.

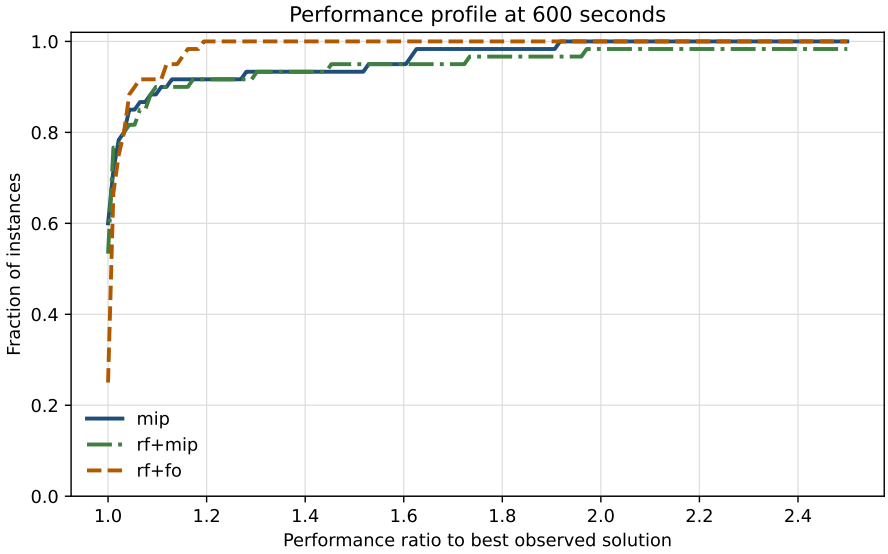

Figure 1: Performance profile at the 600-second budget.

6.8. Figures

Figures 1 and 2 summarize the equal-budget performance profiles, while Figure 3 shows representative convergence trajectories used in the scale-up diagnostics.

7. Conclusions

We revisited a real-world process selection and lot-sizing problem fifteen years after it was reported intractable, confronting a modern MIP solver, temporal-decomposition matheuristics, and a state-of-the-art pure metaheuristic under an equal-resources, pre-registered protocol, on a public benchmark extended to ten problem scales.

Three answers emerge. First, on the practical status of the problem: at the scale of the plant's order book it is simply solved – proven optima in seconds – and this holds up to horizons twice the original, so the first-line recommendation for practitioners is unambiguous: try the monolith with a modern solver before

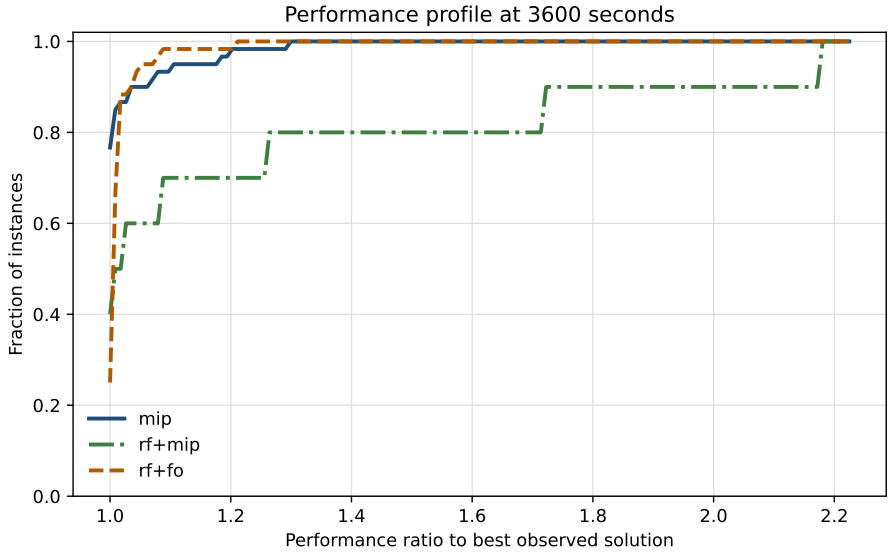

Figure 2: Performance profile at the 3600-second budget.

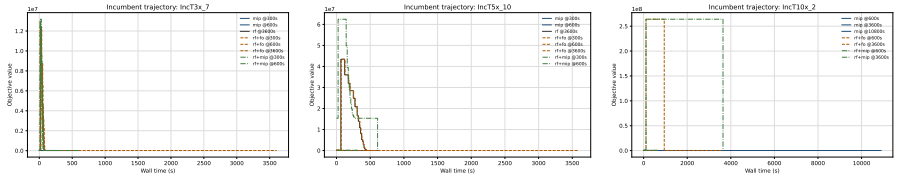

Figure 3: Representative convergence curves for IncT3x_7, IncT5x_10, and IncT10x_2.

building heuristics. What decides difficulty at a given size, however, is not only the size: holding instances, solver, machine, and budget fixed, adding a thousandth-weight penalty on accumulated excess production moves sets 2X–5X from routinely closed to largely open. Modeling choices made for realism deserve the same computational scrutiny as algorithmic ones. Second, on the question of paradigms: when the monolith does falter, what replaces it is not the pure metaheuristic – our strongest GRASP-ILS hybrid, with tenfold parallel resources, is dominated everywhere by MIP-based methods – but decomposition that keeps the solver in the loop. Third, on the question of *when*: the method-choice map has two genuine axes. At equal budgets, decomposition wins the

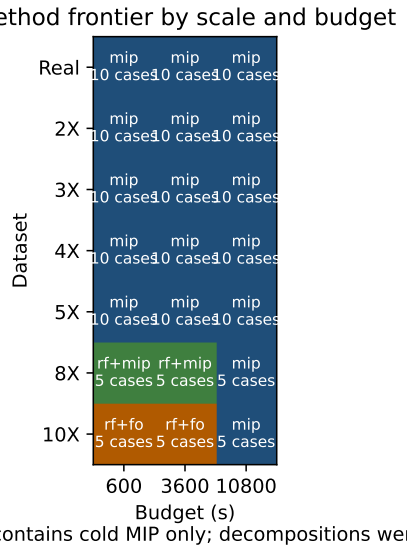

Figure 4: Method-choice frontier by instance family and time budget. Decomposition methods were not executed at 10,800 s; those cells report the cold MIP only and are therefore cross-budget context rather than equal-budget comparisons.

majority of instances from $|T| \approx 150$ upward; granting the monolith triple time restores its lead on most, though not all, of the largest instances. Decomposition thus buys roughly one tier of computing budget – decisive under rolling re-planning deadlines, dispensable when overnight runs are acceptable.

Beyond the verdicts, the study contributes methodological assets: a fully specified, reproducible implementation of relax-and-fix and fix-and-optimize in an algebraic modeling environment, including the budget-guard and construction-floor policies whose absence we showed can silently invalidate comparisons; a 60-instance public benchmark with best-known solutions, dual bounds, trajectories, and per-window logs; and a worked example of pre-registered hypotheses in computational OR, including two registered failures whose diagnoses improved the methods.

Limitations bound the claims. Our comparison with the original study is a statement about practice, not a controlled measurement of solver progress:

solver version, hardware, parallelism, and objective all differ, and disentangling them would require machine-independent protocols we did not attempt. The study covers one industrial sector and deterministic demand; the MFP model prices setups implicitly through the one-process-per-period structure rather than explicitly; horizon replication, while faithful to the inherited benchmark, yields periodic demand; matheuristic construction at extreme scale fell back on a greedy floor, leaving the relax-and-fix window policy suboptimal precisely where decomposition matters most; and decompositions were not run at the largest budget. Future work follows directly: the shortage- and setup-cost extensions of the original study under modern solvers; stochastic order books; and, most immediately, replacing fixed window policies with learned, state-dependent ones – the released per-window logs, containing dual signals, incumbent profiles, and realized improvements for every solve of this campaign, were designed as training data for exactly that line of research.

Acknowledgements

Data availability

The complete benchmark (60 instances in open format), all solution files, and the source code of the matheuristics and analysis pipeline are available at .

References

Achterberg, T., Bixby, R.E., Gu, Z., Rothberg, E., Weninger, D., 2020. Presolve reductions in mixed integer programming. INFORMS Journal on Computing 32, 473–506.

Achterberg, T., Wunderling, R., 2013. Mixed integer programming: analyzing 12 years of progress, in: Facets of Combinatorial Optimization: Festschrift for Martin Grötschel. Springer, pp. 449–481.

Aiex, R.M., Resende, M.G.C., Ribeiro, C.C., 2007. TTT plots: a perl program to create time-to-target plots. Optimization Letters 1, 355–366.

de Araujo, S.A., Arenales, M.N., Clark, A.R., 2008. Lot sizing and furnace scheduling in small foundries. Computers & Operations Research 35, 916–932.

Baker, K.R., 1977. An experimental study of the effectiveness of rolling schedules in production planning. Decision Sciences 8, 19–27.

Bixby, R.E., 2012. A brief history of linear and mixed-integer programming computation. Documenta Mathematica Extra Volume: Optimization Stories, 107–121.

Buschkühl, L., Sahling, F., Helber, S., Tempelmeier, H., 2010. Dynamic capacitated lot-sizing problems: a classification and review of solution approaches. OR Spectrum 32, 231–261.

Clark, A.R., Morabito, R., Toso, E.A.V., 2010. Production setup-sequencing and lot-sizing at an animal nutrition plant through ATSP subtour elimination and patching. Journal of Scheduling 13, 111–121.

Copil, K., Wörbelauer, M., Meyr, H., Tempelmeier, H., 2017. Simultaneous lotsizing and scheduling problems: a classification and review of models. OR Spectrum 39, 1–64. doi:10.1007/s00291-015-0429-4.

Dillenberger, C., Escudero, L.F., Wollensak, A., Zhang, W., 1994. On practical resource allocation for production planning and scheduling with period overlapping setups. European Journal of Operational Research 75, 275–286.

Dolan, E.D., Moré, J.J., 2002. Benchmarking optimization software with performance profiles. Mathematical Programming 91, 201–213.

Drexl, A., Haase, K., 1995. Proportional lotsizing and scheduling. International Journal of Production Economics 40, 73–87.

Drexl, A., Kimms, A., 1997. Lot sizing and scheduling — survey and extensions. European Journal of Operational Research 99, 221–235.

Feo, T.A., Resende, M.G.C., 1995. Greedy randomized adaptive search procedures. Journal of Global Optimization 6, 109–133.

Ferreira, D., Morabito, R., Rangel, S., 2009. Solution approaches for the soft drink integrated production lot sizing and scheduling problem. European Journal of Operational Research 196, 697–706.

Figueira, G., Santos, M.O., Almada-Lobo, B., 2013. A hybrid VNS approach for the short-term production planning and scheduling: a case study in the pulp and paper industry. Computers & Operations Research 40, 1804–1818.

Fischetti, M., Fischetti, M., 2018. Matheuristics, in: Martí, R., Pardalos, P.M., Resende, M.G.C. (Eds.), Handbook of Heuristics. Springer, pp. 121–153.

Fleischmann, B., 1990. The discrete lot-sizing and scheduling problem. European Journal of Operational Research 44, 337–348.

Fleischmann, B., Meyr, H., 1997. The general lotsizing and scheduling problem. OR Spektrum 19, 11–21.

Furlan, M., et al., 2024. Matheuristic for the lot-sizing and scheduling problem in integrated pulp and paper production. Computers & Industrial Engineering TODO complete.

Gleixner, A., et al., 2021. MIPLIB 2017: data-driven compilation of the 6th mixed-integer programming library. Mathematical Programming Computation 13, 443–490. TODO complete author list.

Glock, C.H., Grosse, E.H., Ries, J.M., 2014. The lot sizing problem: a tertiary study. International Journal of Production Economics 155, 39–51. doi:10.1016/j.ijpe.2013.12.009.

Haase, K., 1996. Capacitated lot-sizing with sequence dependent setup costs. OR Spektrum 18, 51–59.

Helber, S., Sahling, F., 2010. A fix-and-optimize approach for the multi-level capacitated lot sizing problem. International Journal of Production Economics 123, 247–256.

James, R.J.W., Almada-Lobo, B., 2011. Single and parallel machine capacitated lotsizing and scheduling: new iterative MIP-based neighborhood search heuristics. Computers & Operations Research 38, 1816–1825.

Jans, R., Degraeve, Z., 2008. Modeling industrial lot sizing problems: a review. International Journal of Production Research 46, 1619–1643.

Karimi, B., Fatemi Ghomi, S.M.T., Wilson, J.M., 2003. The capacitated lot sizing problem: a review of models and algorithms. Omega 31, 365–378.

Karmarkar, U.S., Schrage, L., 1985. The deterministic dynamic product cycling problem. Operations Research 33, 326–345.

Koch, T., Berthold, T., Pedersen, J., Vanaret, C., 2022. Progress in mathematical programming solvers from 2001 to 2020. EURO Journal on Computational Optimization 10, 100031. doi:10.1016/j.ejco.2022.100031.

Laguna, M., Martí, R., 1999. GRASP and path relinking for 2-layer straight line crossing minimization. INFORMS Journal on Computing 11, 44–52.

Leachman, R.C., 2002. Semiconductor production planning, in: Handbook of Applied Optimization. Oxford University Press.

Lourenço, H.R., Martin, O.C., Stützle, T., 2010. Iterated local search: framework and applications, in: Handbook of Metaheuristics. 2nd ed.. Springer, pp. 363–397.

Luche, J.R.D., 2011. Modelos e algoritmos para a otimização do planejamento da produção de grãos eletrofundidos. Tese (doutorado em engenharia de produção). Universidade Federal de São Carlos.

Luche, J.R.D., Morabito, R., Pureza, V., 2009. Combining process selection and lot sizing models for production scheduling of electrofused grains.

Asia-Pacific Journal of Operational Research 26, 421–443. doi:10.1142/S0217595909002286.

Martínez, K.P., Morabito, R., Toso, E.A.V., 2018. A coupled process configuration, lot-sizing and scheduling model for production planning in the molded pulp industry. International Journal of Production Economics 204, 227–243.

Ng, T.S., et al., 2007. Semiconductor production planning using robust optimization, in: Proceedings of the IEEE International Conference on Industrial Engineering and Engineering Management (IEEM). TODO complete author list/pages.

Pochet, Y., Wolsey, L.A., 2006. Production Planning by Mixed Integer Programming. Springer, New York.

Prais, M., Ribeiro, C.C., 2000. Reactive GRASP: an application to a matrix decomposition problem in TDMA traffic assignment. INFORMS Journal on Computing 12, 164–176.

Resende, M.G.C., Ribeiro, C.C., 2016. Optimization by GRASP: Greedy Randomized Adaptive Search Procedures. Springer, New York.

Sahling, F., Buschkühl, L., Tempelmeier, H., Helber, S., 2009. Solving a multi-level capacitated lot sizing problem with multi-period setup carry-over via a fix-and-optimize heuristic. Computers & Operations Research 36, 2546–2553.

Santos, M.O., Almada-Lobo, B., 2012. Integrated pulp and paper mill planning and scheduling. Computers & Industrial Engineering 63, 1–12.

Silva, E.M., Melega, G.M., Akartunalı, K., de Araujo, S.A., 2023. Formulations and theoretical analysis of the one-dimensional multi-period cutting stock problem with setup cost. European Journal of Operational Research TODO volume/pages.

TODO, 2021a. Mathematical modelling and solution approaches for production planning in a chemical industry. Pesquisa Operacional .

TODO, 2021b. MIP approaches for a lot sizing and scheduling problem on multiple production lines with scarce resources, temporary workstations, and perishable products. Journal of the Operational Research Society 72.

Toledo, C.F.M., da Silva Arantes, M., Hossomi, M.Y.B., França, P.M., Akartunalı, K., 2015a. A relax-and-fix with fix-and-optimize heuristic applied to multi-level lot-sizing problems. Journal of Heuristics 21, 687–717. doi:10.1007/s10732-015-9295-0.

Toledo, C.F.M., et al., 2015b. The synchronized and integrated two-level lot sizing and scheduling problem: evaluating the generalized mathematical model. Mathematical Problems in Engineering 2015, 182781. TODO complete author list.

Toscano, A., Ferreira, D., Morabito, R., 2020. Formulation and MIP-heuristics for the lot sizing and scheduling problem with temporal cleanings. Computers & Chemical Engineering 142, 107038. TODO verify volume/pages.

Toso, E.A.V., Morabito, R., Clark, A.R., 2009. Lot sizing and sequencing optimisation at an animal-feed plant. Computers & Industrial Engineering 57, 813–821.

Wörbelauer, M., Meyr, H., Almada-Lobo, B., 2019. Simultaneous lotsizing and scheduling considering secondary resources: a general model, literature review and classification. OR Spectrum 41, 1–43.

Table 4: Method comparison at a 3600-second budget.

Set	Method	Inst.	Mean gap to BKS (%)	Wins	Proven opt.
Real	mip	10	0.00	10	10
Real	rf+fo	10	0.05	5	0
Real	rf+mip	0	–	0	0
Real	ILS v2	10	1.33	1	0
2X	mip	10	0.00	10	10
2X	rf+fo	10	0.45	0	0
2X	rf+mip	0	–	0	0
2X	ILS v2	10	2.87	0	0
3X	mip	10	0.11	9	0
3X	rf+fo	10	1.03	1	0
3X	rf+mip	0	–	0	0
3X	ILS v2	10	12.38	0	0
4X	mip	10	0.54	9	0
4X	rf+fo	10	1.65	1	0
4X	rf+mip	0	–	0	0
4X	ILS v2	10	18.13	0	0
5X	mip	10	2.13	6	0
5X	rf+fo	10	1.54	4	0
5X	rf+mip	0	–	0	0
5X	ILS v2	10	26.88	0	0
8X	mip	5	5.17	0	0
8X	rf+fo	5	5.18	1	0
8X	rf+mip	5	1.49	4	0
8X	ILS v2	0	–	0	0
10X	mip	5	18.02	1	0
10X	rf+fo	5	5.87	3	0
10X	rf+mip	5	15.19	1	0
10X	ILS v2	0	–	0	0

Table 5: Wilcoxon paired tests over matched cells.

Comparison	n	p	Rank-biserial	Median Δ (%)
600s rf+fo vs mip	60	0.2724	0.170	0.12
600s rf+mip vs mip	60	0.3202	0.176	0.00
600s rf+fo vs truncated ILS v2	6	0.0312	-1.000	-20.42
3600s 8X/10X rf+fo vs mip	10	0.1934	-0.491	-4.50
3600s 8X/10X rf+mip vs mip	10	1.0000	-0.018	-1.70

Table 6: Seed variability for the production RF+FO configuration.

Set	Instance	Mean Z	Std.	Min.	Max.
3X	IncT3x_3	46 177.5	0.0	46 177.5	46 177.5
3X	IncT3x_7	16 517.6	0.0	16 517.6	16 517.6
4X	IncT4x_3	50 216.9	0.0	50 216.9	50 216.9
4X	IncT4x_7	19 519.6	0.0	19 519.6	19 519.6
5X	IncT5x_10	16 432.7	448.2	16 115.8	16 749.7
5X	IncT5x_3	56 950.3	0.0	56 950.3	56 950.3
8X	IncT8x_2	135 505.4	330.8	135 271.5	135 739.4
8X	IncT8x_3	77 834.8	106.4	77 759.6	77 910.1
8X	IncT8x_4	160 989.3	731.9	160 471.8	161 506.9
8X	IncT8x_5	60 120.0	195.1	59 982.0	60 258.0
8X	IncT8x_6	96 744.9	237.9	96 576.7	96 913.2
10X	IncT10x_5	80 393.4	3 035.8	78 246.8	82 540.1

Table 7: Method comparison at a 600-second budget.

Set	Method	Inst.	Mean gap to BKS (%)	Wins	Proven opt.
Real	mip	10	0.00	10	0
Real	rf+fo	10	0.05	5	0
Real	rf+mip	10	0.00	10	0
Real	ILS v2	10	3.86	0	0
2X	mip	10	0.06	8	0
2X	rf+fo	10	0.45	0	0
2X	rf+mip	10	0.04	10	0
2X	ILS v2	10	10.50	0	0
3X	mip	10	0.35	5	0
3X	rf+fo	10	1.61	1	0
3X	rf+mip	10	0.13	4	0
3X	ILS v2	10	24.08	0	0
4X	mip	10	3.25	5	0
4X	rf+fo	10	2.75	2	0
4X	rf+mip	10	2.60	3	0
4X	ILS v2	10	29.91	0	0
5X	mip	10	4.42	6	0
5X	rf+fo	10	7.79	2	0
5X	rf+mip	10	5.73	2	0
5X	ILS v2	10	36.32	0	0
8X	mip	5	18.67	1	0
8X	rf+fo	5	14.68	1	0
8X	rf+mip	5	12.13	3	0
8X	ILS v2	0	–	0	0
10X	mip	5	76.62	1	0
10X	rf+fo	5	20.52	4	0
10X	rf+mip	5	108.28	0	0
10X	ILS v2	0	–	0	0

Table 8: Observed method frontier by family and time budget.

Set	Budget (s)	Majority winner	Inst.	mip	rf+fo	rf+mip	Source
Real	600	mip	10	10	0	0	production
Real	3 600	mip	10	10	0	0	registered
Real	10 800	mip	10	10	0	0	registered
2X	600	mip	10	8	0	2	production
2X	3 600	mip	10	10	0	0	registered
2X	10 800	mip	10	10	0	0	registered
3X	600	mip	10	5	1	4	production
3X	3 600	mip	10	9	1	0	registered
3X	10 800	mip	10	10	0	0	registered
4X	600	mip	10	5	2	3	production
4X	3 600	mip	10	9	1	0	registered
4X	10 800	mip	10	10	0	0	registered
5X	600	mip	10	6	2	2	production
5X	3 600	mip	10	6	4	0	registered
5X	10 800	mip	10	10	0	0	registered
8X	600	rf+mip	5	1	1	3	production
8X	3 600	rf+mip	5	1	1	3	post-hoc
8X	10 800	mip	5	5	0	0	post-hoc
10X	600	rf+fo	5	1	4	0	production
10X	3 600	rf+fo	5	1	3	1	post-hoc
10X	10 800	mip	5	5	0	0	post-hoc

Table 9: Cross-budget check: cold MIP at 10800 seconds versus the best decomposition at 3600 seconds.

Set	Instance	mip@10800	Best decomp.@3600	Decomp. method	Δ (%)	Winner
8X	IncT8x_2	128 054.1	131 246.4	rf+mip	-2.43	mip@10800
8X	IncT8x_3	67 597.1	71 896.4	rf+mip	-5.98	mip@10800
8X	IncT8x_4	159 561.6	160 603.0	rf+mip	-0.65	mip@10800
8X	IncT8x_5	59 137.4	58 152.3	rf+mip	1.69	decomp@3600
8X	IncT8x_6	95 679.1	97 667.6	rf+fo	-2.04	mip@10800
10X	IncT10x_2	163 595.5	142 834.2	rf+fo	14.54	decomp@3600
10X	IncT10x_3	80 690.6	87 275.5	rf+mip	-7.54	mip@10800
10X	IncT10x_4	168 985.5	172 982.6	rf+fo	-2.31	mip@10800
10X	IncT10x_5	66 697.0	83 018.4	rf+fo	-19.66	mip@10800
10X	IncT10x_6	111 304.3	109 047.0	rf+fo	2.07	decomp@3600

Table 10: Construction diagnostics for post-fix 8X/10X runs.

Set	Instance	Method	Construction	Z after constr.	RF wall (s)	Improve left (s)	Final Z
8X	IncT8x_2	rf+fo	greedy_floor	59 130 387.1	841.4	2 758.6	135 291.7
8X	IncT8x_2	rf+mip	greedy_floor	58 659 668.2	855.6	2 744.4	131 246.4
8X	IncT8x_3	rf+fo	greedy_floor	63 327 814.3	839.4	2 760.6	77 987.8
8X	IncT8x_3	rf+mip	greedy_floor	63 327 814.3	839.8	2 760.2	71 896.4
8X	IncT8x_4	rf+fo	greedy_floor	60 599 634.0	840.1	2 759.9	160 989.8
8X	IncT8x_4	rf+mip	greedy_floor	60 599 634.0	836.7	2 763.3	160 603.0
8X	IncT8x_5	rf+fo	greedy_floor	59 149 435.4	836.3	2 763.7	60 653.8
8X	IncT8x_5	rf+mip	greedy_floor	59 725 672.9	840.3	2 759.7	58 152.3
8X	IncT8x_6	rf+fo	greedy_floor	53 156 195.9	861.1	2 738.9	97 667.6
8X	IncT8x_6	rf+mip	greedy_floor	57 467 501.0	862.8	2 737.2	99 419.1
10X	IncT10x_2	rf+fo	greedy_floor	64 949 157.2	843.4	2 756.6	142 834.2
10X	IncT10x_2	rf+mip	greedy_floor	64 949 157.2	843.4	2 756.6	154 797.5
10X	IncT10x_3	rf+fo	greedy_floor	50 914 291.2	797.5	2 802.5	88 476.2
10X	IncT10x_3	rf+mip	greedy_floor	50 914 291.2	803.6	2 796.4	87 275.5
10X	IncT10x_4	rf+fo	greedy_floor	53 481 810.0	796.9	2 803.1	172 982.6
10X	IncT10x_4	rf+mip	greedy_floor	62 714 041.0	843.9	2 756.1	376 198.1
10X	IncT10x_5	rf+fo	greedy_floor	52 350 388.5	798.2	2 801.8	83 018.4
10X	IncT10x_5	rf+mip	greedy_floor	43 510 609.0	799.2	2 800.8	87 036.3
10X	IncT10x_6	rf+fo	greedy_floor	53 309 346.3	798.0	2 802.0	109 047.0
10X	IncT10x_6	rf+mip	greedy_floor	63 132 592.9	844.5	2 755.5	187 103.0